

Small Business Cash-Flow Blindspot

System Architecture, Process & Codebase
Computer Engineering Department — PVPIT, Bavdhan, Pune

1. System Architecture

Layered architecture: a client layer (dashboard/API), an orchestration layer that runs the daily pipeline, four core logic modules that map directly to Phases P2-P5 of the implementation plan, a single SQLite data layer, and a config layer that centralises every tunable threshold so business logic never hardcodes constants.

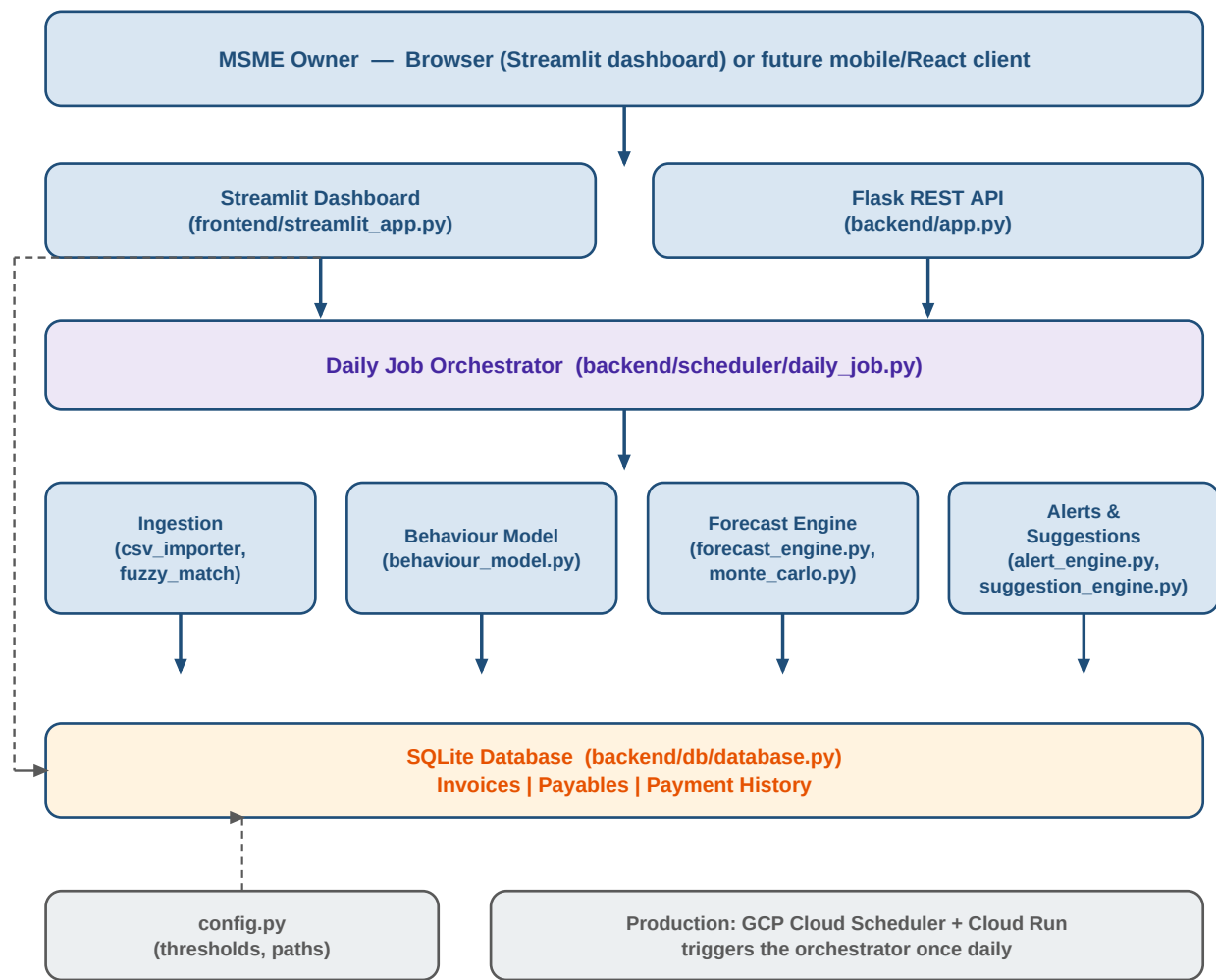


Fig. 1 — Layered architecture. Dashed lines: config feeding the pipeline, and the dashboard's write-back feedback loop into the database.

Layer	Responsibility	Maps to
Client	Renders the dashboard; the only layer the MSME owner interacts with directly.	P6

Layer	Responsibility	Maps to
Interface	Streamlit app for the demo path; Flask API as an alternative for future non-Streamlit clients.	P6
Orchestration	Runs ingestion-independent daily recompute: behaviour model -> forecast -> alerts -> suggestions, in order.	Scheduler
Core logic modules	Four independent, independently-testable modules — ingestion, behaviour model, forecasting, alerts/suggestions.	P1-P5
Data	Single SQLite database with three tables: invoices, payables, payment_history.	P1
Config	One file holding every threshold (shrinkage strength, alert windows, forecast horizon) so tuning never touches logic code.	All layers

2. Complete Process — Request to Response

This traces exactly what runs, in order, from the moment the dashboard loads to the moment the owner sees a number on screen.

#	Call	What it does
1	streamlit_app.py loads	Calls <code>init_db()</code> to ensure tables exist, then reads the sidebar's starting-balance input.
2	<code>run_daily_job(starting_balance)</code>	Single entrypoint into the backend; called fresh on every dashboard load.
3	<code>-> run_forecast()</code>	Pulls unpaid invoices + pending payables from SQLite; calls <code>build_lateness_profiles()</code> internally.
4	<code>-> build_lateness_profiles()</code>	Groups <code>payment_history</code> by customer, applies recency weighting + Bayesian shrinkage, returns one <code>LatenessProfile</code> per customer.
5	<code>-> forecast_engine</code> builds timeline	For each unpaid invoice, shifts its due date by that customer's expected lateness; for each payable, uses its due date as-is; cumulative-sums into a daily balance; finds the first negative day.
6	<code>-> generate_alert(first_shortfall_date)</code>	Converts that date into a tier (critical/warning/info/none) using the windows in <code>config.py</code> .
7	<code>-> generate_suggestions()</code>	Ranks overdue invoices by outstanding-amount x lateness-risk; adds delayable payables (category='discretionary' only).
8	Dashboard renders	Alert banner (colour-coded by tier), a line chart of the balance timeline, and a suggestions table.
9	Owner marks an invoice paid	Writes to <code>invoices</code> + <code>payment_history</code> tables directly from the dashboard; <code>st.rerun()</code> re-triggers step 2 with the updated data.

2.1 Ingestion sub-process (feeds the database before any of the above runs)

#	Step	Detail
1	<code>import_csv(path)</code>	Reads the CSV row by row with pandas.
2	Row type dispatch	Each row is tagged invoice / payable / payment and routed accordingly.
3	<code>resolve_name()</code>	Fuzzy-matches the party name against existing customers/vendors already in the DB (difflib similarity ratio ≥ 0.85) before inserting, so spelling variants collapse to one canonical record.
4	Smart defaults	A missing <code>due_date</code> on an invoice defaults to Net-30 from the issue date.
5	Insert	Cleaned row written to the matching SQLite table.

3. Codebase — Directory Structure

The reference implementation is organised so each folder maps to one phase of the build plan. Every module can be run and tested in isolation.

cashflow_blindspot/

```

README.md
requirements.txt
backend/
  app.py           Flask API endpoint
  config.py        All thresholds & tuning constants
  db/database.py   SQLite schema + connection helper      (P1)
  models/         Invoice, Payable, PaymentHistory dataclasses
  ingestion/
    csv_importer.py  CSV -> cleaned DB rows              (P1)
    fuzzy_match.py   Customer/vendor name resolution      (P1)
  forecasting/
    behaviour_model.py Per-customer lateness estimate      (P2)
    forecast_engine.py Rolling cash-position projection    (P3)
    monte_carlo.py   Shortfall-date confidence band       (P3)
    alerts/alert_engine.py Tiered alert generation        (P4)
  suggestions/
    suggestion_engine.py Ranked chase/delay suggestions    (P5)
  scheduler/daily_job.py Orchestrates P2 -> P5 as one daily run
frontend/
  streamlit_app.py  Dashboard: timeline, alerts, actions    (P6)
data/
  sample_transactions.csv Demo dataset (invoices, payables, payments)
tests/
  test_behaviour_model.py
  test_forecast_engine.py

```

3.1 How the modules call each other

- streamlit_app.py -> scheduler/daily_job.py -> forecast_engine.py -> behaviour_model.py -> db/database.py
- scheduler/daily_job.py -> alerts/alert_engine.py (consumes forecast_engine's output)
- scheduler/daily_job.py -> suggestions/suggestion_engine.py (independent query against db/database.py)
- ingestion/csv_importer.py -> ingestion/fuzzy_match.py + db/database.py (runs before the daily job, not part of it)
- Every module imports its constants from config.py rather than hardcoding values.

3.2 Running it

```

pip install -r requirements.txt
python backend/db/database.py      # creates cashflow.db, loads sample data
streamlit run frontend/streamlit_app.py # opens the dashboard
python -m pytest tests/ -v         # runs the unit tests

```

3.3 What's demo-grade vs. production-grade

Aspect	This codebase (demo)	Production upgrade path
Storage	SQLite, single file	PostgreSQL with indexed rolling views
Scheduling	Called on-demand from the dashboard	GCP Cloud Scheduler + Cloud Run, once daily

Aspect	This codebase (demo)	Production upgrade path
Fuzzy matching	diffliib (stdlib, zero dependencies)	rapidfuzz, for larger customer lists
Frontend	Streamlit (fast to demo)	Flask/React (app.py already exposes the same logic over REST)
Auth & security	None (single-user demo)	At-rest encryption, per-business data isolation, auth layer

Prepared for academic project submission — Computer Engineering Department, PVPIT.